2 Developing web applications

This chapter covers

- Presenting model data in the browser
- Processing and validating form input
- Choosing a view template library

First impressions are important. Curb appeal can sell a house long before the home buyer enters the door. A car's cherry red paint job will turn more heads than what's under the hood. And literature is replete with stories of love at first sight. What's inside is important, but what's outside—what's seen first—-is also important.

The applications you'll build with Spring will do all kinds of things, including crunching data, reading information from a database, and interacting with other applications. But the first impression your application users will get comes from the user interface. And in many applications, that UI is a web application presented in a browser.

In chapter 1, you created your first Spring MVC controller to display your application home page. But Spring MVC can do far more than simply display static content. In this chapter, you'll develop the first major bit of functionality in your Taco Cloud application —the ability to design custom tacos. In doing so, you'll dig deeper into Spring MVC, and you'll see how to display model data and process form input.

2.1 Displaying information

Fundamentally, Taco Cloud is a place where you can order tacos online. But more than that, Taco Cloud wants to enable its customers to express their creative side and design custom tacos from a rich palette of ingredients.

Therefore, the Taco Cloud web application needs a page that displays the selection of ingredients for taco artists to choose from. The ingredient choices may change at any time, so they shouldn't be hardcoded into an HTML page. Rather, the list of available ingredients should be fetched from a database and handed over to the page to be displayed to the customer.

In a Spring web application, it's a controller's job to fetch and process data. And it's a view's job to render that data into HTML that will be displayed in the browser. You're going to create the following components in support of the taco creation page:

- A domain class that defines the properties of a taco ingredient
- A Spring MVC controller class that fetches ingredient information and passes it along to the view
- A view template that renders a list of ingredients in the user's browser

The relationship between these components is illustrated in figure 2.1.

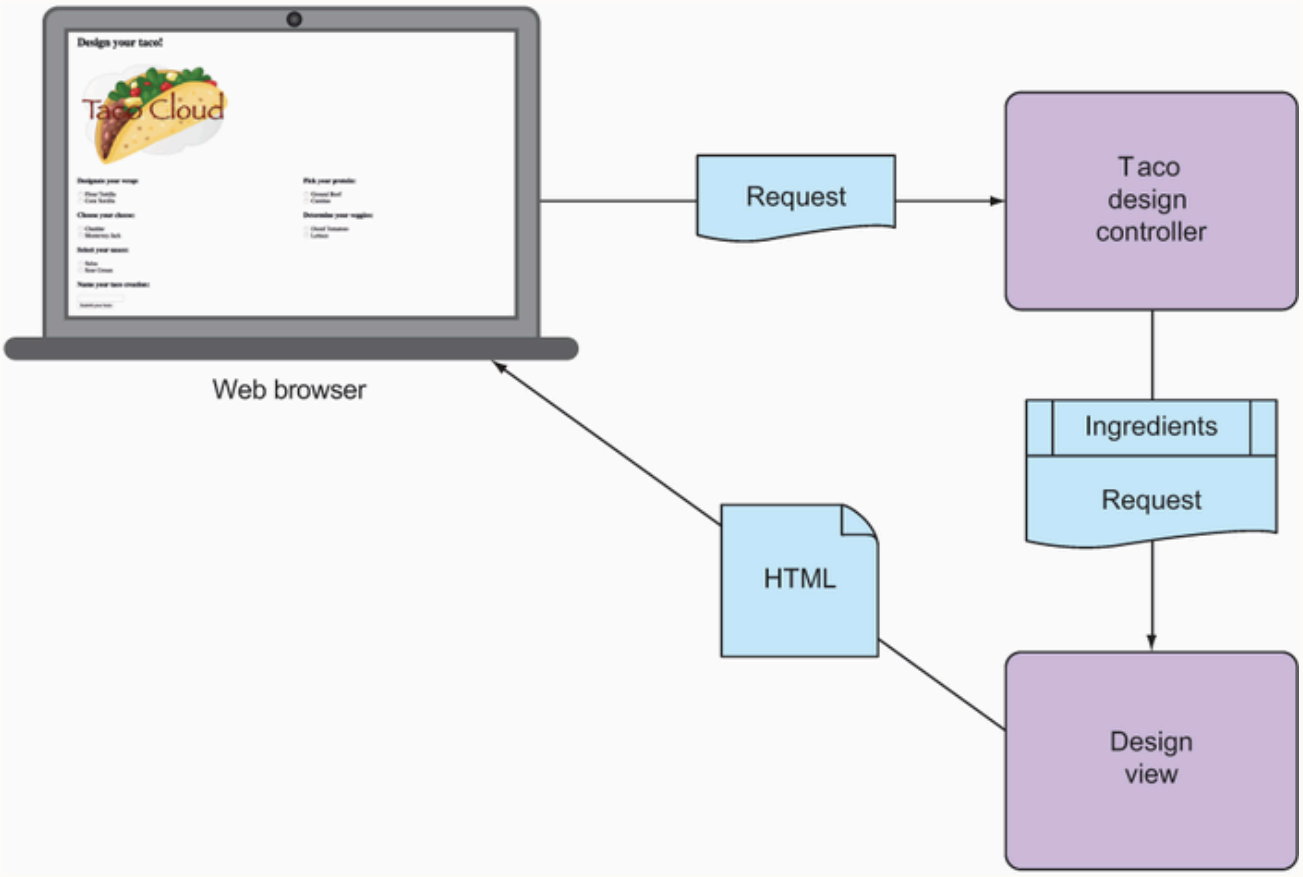


Figure 2.1 A typical Spring MVC request flow

Because this chapter focuses on Spring's web framework, we'll defer any of the database stuff to chapter 3. For now, the controller is solely responsible for providing the ingredients to the view. In chapter 3, you'll rework the controller to collaborate with a repository that fetches ingredients data from a database.

Before you write the controller and view, let's hammer out the domain type that represents an ingredient. This will establish a foundation on which you can develop your web components.

1.1 Establishing the domain

An application's domain is the subject area that it addresses—the ideas and concepts that influence the understanding of the application.[1] In the Taco Cloud application, the domain includes such objects as taco designs, the ingredients that those designs are composed of, customers, and taco orders placed by the customers. Figure 2.2 shows these entities and how they are related.

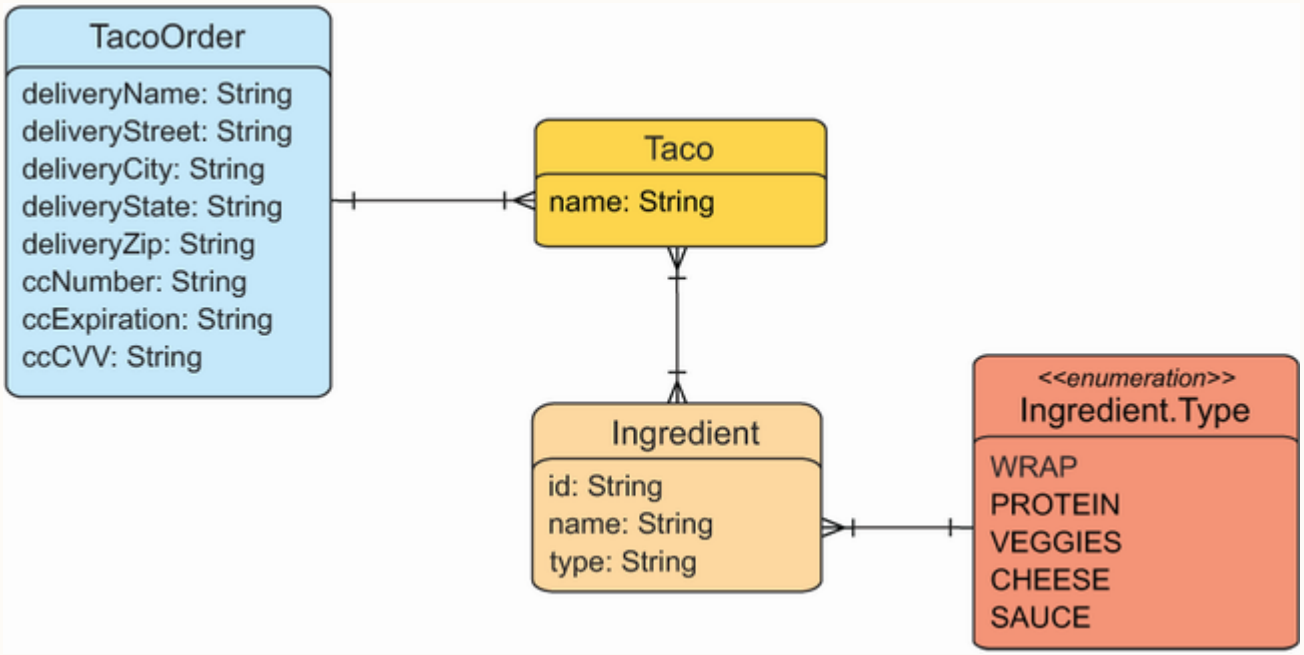


Figure 2.2 The Taco Cloud domain

To get started, we'll focus on taco ingredients. In your domain, taco ingredients are fairly simple objects. Each has a name as well as a type so that it can be visually categorized (proteins, cheeses, sauces, and so on). Each also has an ID by which it can easily and unambiguously be referenced. The following `Ingredient` class defines the domain object you need.

Listing 2.1 Defining taco ingredients

```
package tacos;

import lombok.Data;

@Data
public class Ingredient {

  private final String id;
  private final String name;
  private final Type type;
```

```
  public enum Type {
    WRAP, PROTEIN, VEGGIES, CHEESE, SAUCE
  }

}
```

As you can see, this is a run-of-the-mill Java domain class, defining the three properties needed to describe an ingredient. Perhaps the most unusual thing about the `Ingredient` class as defined in listing 2.1 is that it seems to be missing the usual set of getter and setter methods, not to mention useful methods like `equals()`, `hashCode()`, `toString()`, and others.

You don't see them in the listing partly to save space, but also because you're using an amazing library called Lombok to automatically generate those methods at compile time so that they will be available at run time. In fact, the `@Data` annotation at the class level is provided by Lombok and tells Lombok to generate all of those missing methods as well as a constructor that accepts all `final` properties as arguments. By using Lombok, you can keep the code for `Ingredient` slim and trim.

Lombok isn't a Spring library, but it's so incredibly useful that I find it hard to develop without it. Plus, it's a lifesaver when I need to keep code examples in a book short and sweet.

To use Lombok, you'll need to add it as a dependency in your project. If you're using Spring Tool Suite, it's an easy matter of right-clicking on the pom.xml file and selecting Add Starters from the Spring context menu. The same selection of dependencies you were given in chapter 1 (in figure 1.4) will appear, giving you a chance to add or change your selected dependencies. Find Lombok under Developer Tools, make sure it's selected, and click OK; Spring Tool Suite automatically adds it to your build specification.

Alternatively, you can manually add it with the following entry in pom.xml:

```
<dependency>
  <groupId>org.projectlombok</groupId>
  <artifactId>lombok</artifactId>
</dependency>
```

If you decide to manually add Lombok to your build, you'll also want to exclude it from the Spring Boot Maven plugin in the `<build>` section of the pom.xml file:

```
<build>
 <plugins>
  <plugin>
   <groupId>org.springframework.boot</groupId>
   <artifactId>spring-boot-maven-plugin</artifactId>
   <configuration>
    <excludes>
     <exclude>
      <groupId>org.projectlombok</groupId>
      <artifactId>lombok</artifactId>
     </exclude>
    </excludes>
   </configuration>
  </plugin>
 </plugins>
</build>
```

Lombok's magic is applied at compile time, so there's no need for it to be available at run time. Excluding it like this keeps it out of the resulting JAR or WAR file.

The Lombok dependency provides you with Lombok annotations (such as `@Data`) at development time and with automatic method generation at compile time. But you'll also need to add Lombok as an extension in your IDE, or your IDE will complain, with errors about missing methods and `final` properties that aren't being set. Visit **https://projectlombok.org/** to find out how to install Lombok in your IDE of choice.

Why are there so many errors in my code?

It bears repeating that when using Lombok, you **must** install the Lombok plugin into your IDE. Without it, your IDE won't be aware that Lombok is providing getters, setters, and other methods and will complain that they are missing.

Lombok is supported in a number of popular IDEs, including Eclipse, Spring Tool Suite, IntelliJ IDEA, and Visual Studio Code. Visit **https://projectlombok.org/** for more information on how to install the Lombok plugin into your IDE.

I think you'll find Lombok to be very useful, but know that it's optional. You don't need it to develop Spring applications, so if you'd rather not use it, feel free to write those missing methods by hand. Go ahead ... I'll wait.

Ingredients are the essential building blocks of a taco. To capture how those ingredients are brought together, we'll define the `Taco` domain class, as shown next.

Listing 2.2 A domain object defining a taco design

```
package tacos;
import java.util.List;
import lombok.Data;

@Data
public class Taco {

  private String name;

  private List<Ingredient> ingredients;

}
```

As you can see, `Taco` is a straightforward Java domain object with a couple of properties. Like `Ingredient`, the `Taco` class is annotated with `@Data` to have Lombok automatically generate essential JavaBean methods for you at compile time.

Now that we have defined `Ingredient` and `Taco`, we need one more domain class that defines how customers specify the tacos that they want to order, along with payment and delivery information. That's the job of the `TacoOrder` class, shown here.

Listing 2.3 A domain object for taco orders

```
package tacos;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
public class TacoOrder {

  private String deliveryName;
  private String deliveryStreet;
  private String deliveryCity;
  private String deliveryState;
  private String deliveryZip;
  private String ccNumber;
  private String ccExpiration;
  private String ccCVV;

  private List<Taco> tacos = new ArrayList<>();
```

```
  public void addTaco(Taco taco) {
    tacos.add(taco);
  }
}
```

Aside from having more properties than either `Ingredient` or `Taco`, there's nothing particularly new to discuss about `TacoOrder`. It's a simple domain class with nine properties: five for delivery information, three for payment information, and one that is the list of `Taco` objects that make up the order. There's also an `addTaco()` method that's added for the convenience of adding tacos to the order.

Now that the domain types are defined, we're ready to put them to work. Let's add a few controllers to handle web requests in the application.

1.2 Creating a controller class

Controllers are the major players in Spring's MVC framework. Their primary job is to handle HTTP requests and either hand off a request to a view to render HTML (browser-displayed) or write data directly to the body of a response (RESTful). In this chapter, we're focusing on the kinds of controllers that use views to produce content for web browsers. When we get to chapter 7, we'll look at writing controllers that handle requests in a REST API.

For the Taco Cloud application, you need a simple controller that will do the following:

- Handle HTTP `GET` requests where the request path is /design
- Build a list of ingredients
- Hand off the request and the ingredient data to a view template to be rendered as HTML and sent to the requesting web browser

The `DesignTacoController` class in the next listing addresses those requirements.

Listing 2.4 The beginnings of a Spring controller class

```
package tacos.web;

import java.util.Arrays;
import java.util.List;
import java.util.stream.Collectors;
import org.springframework.stereotype.Controller;
import org.springframework.ui.Model;
```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.ModelAttribute;
import org.springframework.web.bind.annotation.PostMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;

import lombok.extern.slf4j.Slf4j;
import tacos.Ingredient;
import tacos.Ingredient.Type;
import tacos.Taco;
import tacos.TacoOrder

@Slf4j
@Controller
@RequestMapping("/design")
@SessionAttributes("tacoOrder")
public class DesignTacoController {

@ModelAttribute
public void addIngredientsToModel(Model model) {
   List<Ingredient> ingredients = Arrays.asList(
     new Ingredient("FLTO", "Flour Tortilla", Type.WRAP),
     new Ingredient("COTO", "Corn Tortilla", Type.WRAP),
     new Ingredient("GRBF", "Ground Beef", Type.PROTEIN),
     new Ingredient("CARN", "Carnitas", Type.PROTEIN),
     new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES),
     new Ingredient("LETC", "Lettuce", Type.VEGGIES),
     new Ingredient("CHED", "Cheddar", Type.CHEESE),
     new Ingredient("JACK", "Monterrey Jack", Type.CHEESE),
     new Ingredient("SLSA", "Salsa", Type.SAUCE),
     new Ingredient("SRCR", "Sour Cream", Type.SAUCE)
   );

   Type[] types = Ingredient.Type.values();
   for (Type type : types) {
     model.addAttribute(type.toString().toLowerCase(),
         filterByType(ingredients, type));
   }
 }

 @ModelAttribute(name = "tacoOrder")
 public TacoOrder order() {
   return new TacoOrder();
 }

 @ModelAttribute(name = "taco")
```

```
  public Taco taco() {
    return new Taco();
  }

  @GetMapping
  public String showDesignForm() {
    return "design";
  }

  private Iterable<Ingredient> filterByType(
      List<Ingredient> ingredients, Type type) {
    return ingredients
            .stream()
            .filter(x -> x.getType().equals(type))
            .collect(Collectors.toList());
  }

}
```

The first thing to note about `DesignTacoController` is the set of annotations applied at the class level. The first, `@Slf4j`, is a Lombok-provided annotation that, at compilation time, will automatically generate an SLF4J (Simple Logging Facade for Java, https://www.slf4j.org/) `Logger` static property in the class. This modest annotation has the same effect as if you were to explicitly add the following lines within the class:

```
private static final org.slf4j.Logger log =
    org.slf4j.LoggerFactory.getLogger(DesignTacoController.class);
```

You'll make use of this `Logger` a little later.

The next annotation applied to `DesignTacoController` is `@Controller`. This annotation serves to identify this class as a controller and to mark it as a candidate for component scanning, so that Spring will discover it and automatically create an instance of `DesignTacoController` as a bean in the Spring application context.

`DesignTacoController` is also annotated with `@RequestMapping`. The `@RequestMapping` annotation, when applied at the class level, specifies the kind of requests that this controller handles. In this case, it specifies that `DesignTacoController` will handle requests whose path begins with /design.

Finally, you see that `DesignTacoController` is annotated with `@SessionAttributes("tacoOrder")`. This indicates that the `TacoOrder` object that is put into the model a little later in the class should be maintained in session. This is important because the creation of a taco is also the first step in creating an order, and the order we create will need to be carried in the session so that it can span multiple requests.

HANDLING A GET REQUEST

The class-level `@RequestMapping` specification is refined with the `@GetMapping` annotation that adorns the `showDesignForm()` method. `@GetMapping`, paired with the class-level `@RequestMapping`, specifies that when an HTTP `GET` request is received for /design, Spring MVC will call `showDesignForm()` to handle the request.

`@GetMapping` is just one member of a family of request-mapping annotations. Table 2.1 lists all of the request-mapping annotations available in Spring MVC.

Table 2.1 Spring MVC request-mapping annotations

Annotation	Description
@RequestMapping	General-purpose request handling
@GetMapping	Handles HTTP `GET` requests
@PostMapping	Handles HTTP `POST` requests
@PutMapping	Handles HTTP `PUT` requests
@DeleteMapping	Handles HTTP `DELETE` requests
@PatchMapping	Handles HTTP `PATCH` requests

When `showDesignForm()` handles a `GET` request for /design, it doesn't really do much. The main thing it does is return a `String` value of `"taco"`, which is the logical name of the view that will be used to render the model to the browser. But before it does that, it also populates the given `Model` with an empty `Taco` object under a key whose name is `"taco"`. This will enable the form to have a blank slate on which to create a taco masterpiece.

It would seem that a `GET` request to /design doesn't do much. But on the contrary, there's a bit more involved than what is found in the `showDesignForm()` method. You'll also notice a method named `addIngredientsToModel()` that is annotated with `@ModelAttribute`. This method will also be invoked when a request is handled and will construct a list of `Ingredient` objects to be put into the model. The list is hardcoded for now. When we get to chapter 3, you'll pull the list of available taco ingredients from a database.

Once the list of ingredients is ready, the next few lines of `addIngredientsToModel()` filters the list by ingredient type using a helper method named `filterByType()`. A list of ingredient types is then added as an attribute to the `Model` object that will be passed into `showDesignForm()`. `Model` is an object that ferries data between a controller and whatever view is charged with rendering that data. Ultimately, data that's placed in `Model` attributes is copied into the servlet request attributes, where the view can find them and use them to render a page in the user's browser.

Following `addIngredientsToModel()` are two more methods that are also annotated with `@ModelAttribute`. These methods are much simpler and create only a new `TacoOrder` and `Taco` object to place into the model. The `TacoOrder` object, referred to earlier in the `@SessionAttributes` annotation, holds state for the order being built as the user creates tacos across multiple requests. The `Taco` object is placed into the model so that the view rendered in response to the `GET` request for /design will have a non-`null` object to display.

Your `DesignTacoController` is really starting to take shape. If you were to run the application now and point your browser at the /design path, the `DesignTacoController`'s `showDesignForm()` and `addIngredientsToModel()` would be engaged, placing ingredients and an empty `Taco` into the model before passing the request on to the view. But because you haven't defined the view yet, the request would take a horrible turn, resulting in an HTTP 500 (Internal Server Error) error. To fix that, let's switch our attention to the view where the data will be decorated with HTML to be presented in the user's web browser.

1.3 Designing the view

After the controller is finished with its work, it's time for the view to get going. Spring offers several great options for defining views, including JavaServer Pages (JSP), Thymeleaf, FreeMarker, Mustache, and Groovy-based templates. For now, we'll use Thymeleaf, the choice we made in chapter 1 when starting the project. We'll consider a few of the other options in section 2.5.

We have already added Thymeleaf as a dependency in chapter 1. At run time, Spring Boot autoconfiguration sees that Thymeleaf is in the classpath and automatically creates the beans that support Thymeleaf views for Spring MVC.

View libraries such as Thymeleaf are designed to be decoupled from any particular web framework. As such, they're unaware of Spring's model abstraction and are unable to work with the data that the controller places in `Model`. But they can work with servlet request attributes. Therefore, before Spring hands the request over to a view, it copies the model data into request attributes that Thymeleaf and other view-templating options have ready access to.

Thymeleaf templates are just HTML with some additional element attributes that guide a template in rendering request data. For example, if there were a request attribute whose key is `"message"`, and you wanted it to be rendered into an HTML `<p>` tag by Thymeleaf, you'd write the following in your Thymeleaf template:

```
<p th:text="${message}">placeholder message</p>
```

When the template is rendered into HTML, the body of the `<p>` element will be replaced with the value of the servlet request attribute whose key is `"message"`. The `th:text` attribute is a Thymeleaf namespace attribute that performs the replacement. The `${}` operator tells it to use the value of a request attribute (`"message"`, in this case).

Thymeleaf also offers another attribute, `th:each`, that iterates over a collection of elements, rendering the HTML once for each item in the collection. This attribute will come in handy as you design your view to list taco ingredients from the model. For example, to render just the list of `"wrap"` ingredients, you can use the following snippet of HTML:

```
<h3>Designate your wrap:</h3>
<div th:each="ingredient : ${wrap}">
  <input th:field="*{ingredients}" type="checkbox"
         th:value="${ingredient.id}"/>
  <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
```

Here, you use the `th:each` attribute on the `<div>` tag to repeat rendering of the `<div>` once for each item in the collection found in the `wrap` request attribute. On each iteration, the ingredient item is bound to a Thymeleaf variable named `ingredient`.

Inside the `<div>` element are a check box `<input>` element and a `<span>` element to provide a label for the check box. The check box uses Thymeleaf's `th:value` to set the rendered `<input>` element's `value` attribute to the value found in the ingredient's `id` property. The `th:field` attribute ultimately sets the `<input>` element's `name` attribute and is used to remember whether or not the check box is checked. When we add validation later, this will ensure that the check box maintains its state should the form need to be redisplayed after a validation error. The `<span>` element uses `th:text` to replace the `"INGREDIENT"` placeholder text with the value of the ingredient's `name` property.

When rendered with actual model data, one iteration of that `<div>` loop might look like this:

```
<div>
  <input name="ingredients" type="checkbox" value="FLTO" />
  <span>Flour Tortilla</span><br/>
</div>
```

Ultimately, the preceding Thymeleaf snippet is just part of a larger HTML form through which your taco artist users will submit their tasty creations. The complete Thymeleaf template, including all ingredient types and the form, is shown in the following listing.

Listing 2.5 The complete design-a-taco page

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>

  <body>
    <h1>Design your taco!</h1>
    <img th:src="@{/images/TacoCloud.png}"/>

    <form method="POST" th:object="${taco}">
    <div class="grid">
      <div class="ingredient-group" id="wraps">
      <h3>Designate your wrap:</h3>
      <div th:each="ingredient : ${wrap}">
        <input th:field="*{ingredients}" type="checkbox"
               th:value="${ingredient.id}"/>
```

```
 <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
</div>

<div class="ingredient-group" id="proteins">
<h3>Pick your protein:</h3>
<div th:each="ingredient : ${protein}">
 <input th:field="*{ingredients}" type="checkbox"
       th:value="${ingredient.id}"/>
 <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
</div>

<div class="ingredient-group" id="cheeses">
<h3>Choose your cheese:</h3>
<div th:each="ingredient : ${cheese}">
 <input th:field="*{ingredients}" type="checkbox"
       th:value="${ingredient.id}"/>
 <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
</div>

<div class="ingredient-group" id="veggies">
<h3>Determine your veggies:</h3>
<div th:each="ingredient : ${veggies}">
 <input th:field="*{ingredients}" type="checkbox"
       th:value="${ingredient.id}"/>
 <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
</div>

<div class="ingredient-group" id="sauces">
<h3>Select your sauce:</h3>
<div th:each="ingredient : ${sauce}">
 <input th:field="*{ingredients}" type="checkbox"
       th:value="${ingredient.id}"/>
 <span th:text="${ingredient.name}">INGREDIENT</span><br/>
</div>
</div>
</div>

<div>


<h3>Name your taco creation:</h3>
<input type="text" th:field="*{name}"/>
```

```
      <br/>

      <button>Submit Your Taco</button>
      </div>
    </form>
  </body>
</html>
```

As you can see, you repeat the `<div>` snippet for each of the types of ingredients, and you include a Submit button and field where the user can name their creation.

It's also worth noting that the complete template includes the Taco Cloud logo image and a `<link>` reference to a stylesheet.[2] In both cases, Thymeleaf's `@{}` operator is used to produce a context-relative path to the static artifacts that these tags are referencing. As you learned in chapter 1, static content in a Spring Boot application is served from the /static directory at the root of the classpath.

Now that your controller and view are complete, you can fire up the application to see the fruits of your labor. We have many ways to run a Spring Boot application. In chapter 1, I showed you how to run the application by clicking the Start button in the Spring Boot Dashboard. No matter how you fire up the Taco Cloud application, once it starts, point your browser to http://localhost:8080/design. You should see a page that looks something like figure 2.3.

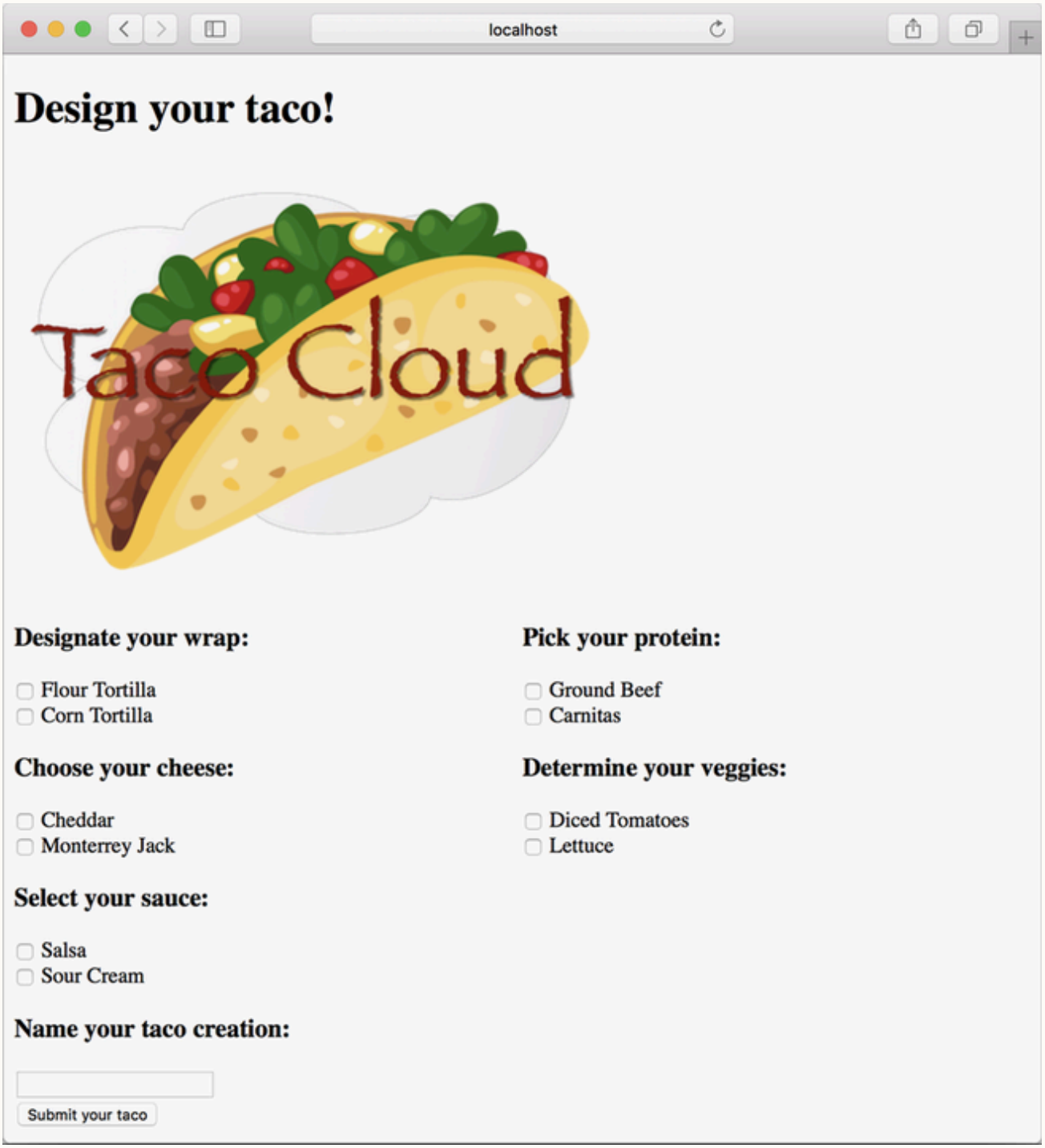


Figure 2.3 The rendered taco design page

It's looking good! A taco artist visiting your site is presented with a form containing a palette of taco ingredients from which they can create their masterpiece. But what happens when they click the Submit Your Taco button?

Your `DesignTacoController` isn't yet ready to accept taco creations. If the design form is submitted, the user will be presented with an error. (Specifically, it will be an HTTP 405 error: Request Method "POST" Not Supported.) Let's fix that by writing some more controller code that handles form submission.

2.2 Processing form submission

If you take another look at the `<form>` tag in your view, you can see that its `method` attribute is set to `POST`. Moreover, the `<form>` doesn't declare an `action` attribute. This means that when the form is submitted, the browser will gather all the data in the form and send it to the server in an HTTP `POST` request to the same path for which a `GET` request displayed the form—the /design path.

Therefore, you need a controller handler method on the receiving end of that `POST` request. You need to write a new handler method in `DesignTacoController` that handles a `POST` request for /design.

In listing 2.4, you used the `@GetMapping` annotation to specify that the `showDesignForm()` method should handle HTTP `GET` requests for /design. Just like `@GetMapping` handles `GET` requests, you can use `@PostMapping` to handle `POST` requests. For handling taco design submissions, add the `processTaco()` method in the following listing to `DesignTacoController`.

Listing 2.6 Handling `POST` requests with `@PostMapping`

```
@PostMapping
public String processTaco(Taco taco,
        @ModelAttribute TacoOrder tacoOrder) {
  tacoOrder.addTaco(taco);
  log.info("Processing taco: {}", taco);

  return "redirect:/orders/current";
}
```

As applied to the `processTaco()` method, `@PostMapping` coordinates with the class-level `@RequestMapping` to indicate that `processTaco()` should handle `POST` requests for /design. This is precisely what you need to process a taco artist's submitted creations.

When the form is submitted, the fields in the form are bound to properties of a `Taco` object (whose class is shown in the next listing) that's passed as a parameter into `processTaco()`. From there, the `processTaco()` method can do whatever it wants with the `Taco` object. In this case, it adds the `Taco` to the `TacoOrder` object passed as a parameter to the method and then logs it. The `@ModelAttribute` applied to the `TacoOrder` parameter indicates that it should use the `TacoOrder` object that was placed into the model via the `@ModelAttribute`-annotated `order()` method shown earlier in listing 2.4.

If you look back at the form in listing 2.5, you'll see several `checkbox` elements, all with the name `ingredients`, and a text input element named `name`. Those fields in the form correspond directly to the `ingredients` and `name` properties of the `Taco` class.

The `name` field on the form needs to capture only a simple textual value. Thus the `name` property of `Taco` is of type `String`. The ingredients check boxes also have textual values, but because zero or many of them may be selected, the `ingredients` property that they're bound to is a `List<Ingredient>` that will capture each of the chosen ingredients.

But wait. If the ingredients check boxes have textual (e.g., `String`) values, but the `Taco` object represents a list of ingredients as `List<Ingredient>`, then isn't there a mismatch? How can a textual list like `["FLTO", "GRBF", "LETC"]` be bound to a list of `Ingredient` objects that are richer objects containing not only an ID but also a descriptive name and ingredient type?

That's where a converter comes in handy. A converter is any class that implements Spring's `Converter` interface and implements its `convert()` method to take one value and convert it to another. To convert a `String` to an `Ingredient`, we'll use the `IngredientByIdConverter` as follows.

Listing 2.7 Converting strings to ingredients

```
package tacos.web;

import java.util.HashMap;
import java.util.Map;
import org.springframework.core.convert.converter.Converter;
import org.springframework.stereotype.Component;

import tacos.Ingredient;
import tacos.Ingredient.Type;

@Component
public class IngredientByIdConverter implements Converter<String, Ingredient> {

  private Map<String, Ingredient> ingredientMap = new HashMap<>();

  public IngredientByIdConverter() {
    ingredientMap.put("FLTO",
        new Ingredient("FLTO", "Flour Tortilla", Type.WRAP));
    ingredientMap.put("COTO",
        new Ingredient("COTO", "Corn Tortilla", Type.WRAP));
```

```
    ingredientMap.put("GRBF",
        new Ingredient("GRBF", "Ground Beef", Type.PROTEIN));
    ingredientMap.put("CARN",
        new Ingredient("CARN", "Carnitas", Type.PROTEIN));
    ingredientMap.put("TMTO",
        new Ingredient("TMTO", "Diced Tomatoes", Type.VEGGIES));
    ingredientMap.put("LETC",
        new Ingredient("LETC", "Lettuce", Type.VEGGIES));
    ingredientMap.put("CHED",
        new Ingredient("CHED", "Cheddar", Type.CHEESE));
    ingredientMap.put("JACK",
        new Ingredient("JACK", "Monterrey Jack", Type.CHEESE));
    ingredientMap.put("SLSA",
        new Ingredient("SLSA", "Salsa", Type.SAUCE));
    ingredientMap.put("SRCR",
        new Ingredient("SRCR", "Sour Cream", Type.SAUCE));
  }

  @Override
  public Ingredient convert(String id) {
    return ingredientMap.get(id);
  }

}
```

Because we don't yet have a database from which to pull `Ingredient` objects, the constructor of `IngredientByIdConverter` creates a `Map` keyed on a `String` that is the ingredient ID and whose values are `Ingredient` objects. In chapter 3, we'll adapt this converter to pull the ingredient data from a database instead of being hardcoded like this. The `convert()` method then simply takes a `String` that is the ingredient ID and uses it to look up the `Ingredient` from the map.

Notice that the `IngredientByIdConverter` is annotated with `@Component` to make it discoverable as a bean in the Spring application context. Spring Boot autoconfiguration will discover this, and any other `Converter` beans, and will automatically register them with Spring MVC to be used when the conversion of request parameters to bound properties is needed.

For now, the `processTaco()` method does nothing with the `Taco` object. In fact, it doesn't do much of anything at all. That's OK. In chapter 3, you'll add some persistence logic that will save the submitted `Taco` to a database.

Just as with the `showDesignForm()` method, `processTaco()` finishes by returning a `String` value. And just like `showDesignForm()`, the value returned indicates a view that will be shown to the user. But what's different is that the value returned from `processTaco()` is prefixed with `"redirect:"`, indicating that this is a redirect view. More specifically, it indicates that after `processTaco()` completes, the user's browser should be redirected to the relative path /orders/current.

The idea is that after creating a taco, the user will be redirected to an order form from which they can place an order to have their taco creations delivered. But you don't yet have a controller that will handle a request for /orders/current.

Given what you now know about `@Controller`, `@RequestMapping`, and `@GetMapping`, you can easily create such a controller. It might look something like the following listing.

Listing 2.8 A controller to present a taco order form

```
package tacos.web;
import org.springframework.stereotype.Controller;
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.SessionAttributes;
import org.springframework.web.bind.support.SessionStatus;

import lombok.extern.slf4j.Slf4j;
import tacos.TacoOrder;

@Slf4j
@Controller
@RequestMapping("/orders")
@SessionAttributes("tacoOrder")
public class OrderController {

  @GetMapping("/current")
  public String orderForm() {
    return "orderForm";
  }

}
```

Once again, you use Lombok's `@Slf4j` annotation to create a free SLF4J `Logger` object at compile time. You'll use this `Logger` in a moment to log the details of the order that's

submitted.

The class-level `@RequestMapping` specifies that any request-handling methods in this controller will handle requests whose path begins with /orders. When combined with the method-level `@GetMapping`, it specifies that the `orderForm()` method will handle HTTP `GET` requests for /orders/current.

As for the `orderForm()` method itself, it's extremely basic, only returning a logical view name of `orderForm`. Once you have a way to persist taco creations to a database in chapter 3, you'll revisit this method and modify it to populate the model with a list of `Taco` objects to be placed in the order.

The `orderForm` view is provided by a Thymeleaf template named orderForm.html, which is shown next.

Listing 2.9 A taco order form view

```
<!DOCTYPE html>
<html xmlns="http://www.w3.org/1999/xhtml"
      xmlns:th="http://www.thymeleaf.org">
  <head>
    <title>Taco Cloud</title>
    <link rel="stylesheet" th:href="@{/styles.css}" />
  </head>

  <body>

    <form method="POST" th:action="@{/orders}" th:object="${tacoOrder}">
      <h1>Order your taco creations!</h1>

      <img th:src="@{/images/TacoCloud.png}"/>

      <h3>Your tacos in this order:</h3>
      <a th:href="@{/design}" id="another">Design another taco</a><br/>
      <ul>
        <li th:each="taco : ${tacoOrder.tacos}">
          <span th:text="${taco.name}">taco name</span></li>
      </ul>

      <h3>Deliver my taco masterpieces to...</h3>
      <label for="deliveryName">Name: </label>
      <input type="text" th:field="*{deliveryName}"/>
      <br/>
```

```
      <label for="deliveryStreet">Street address: </label>
      <input type="text" th:field="*{deliveryStreet}"/>
      <br/>

      <label for="deliveryCity">City: </label>
      <input type="text" th:field="*{deliveryCity}"/>
      <br/>

      <label for="deliveryState">State: </label>
      <input type="text" th:field="*{deliveryState}"/>
      <br/>

      <label for="deliveryZip">Zip code: </label>
      <input type="text" th:field="*{deliveryZip}"/>
      <br/>

      <h3>Here's how I'll pay...</h3>
      <label for="ccNumber">Credit Card #: </label>
      <input type="text" th:field="*{ccNumber}"/>
      <br/>

      <label for="ccExpiration">Expiration: </label>
      <input type="text" th:field="*{ccExpiration}"/>
      <br/>

      <label for="ccCVV">CVV: </label>
      <input type="text" th:field="*{ccCVV}"/>
      <br/>

      <input type="submit" value="Submit Order"/>
    </form>
  </body>
</html>
```

For the most part, the orderForm.html view is typical HTML/Thymeleaf content, with very little of note. It starts by listing the tacos that were added to the order. It uses Thymeleaf's `th:each` to cycle through the order's `tacos` property as it creates the list. Then it renders the order form.

But notice that the `<form>` tag here is different from the `<form>` tag used in listing 2.5 in that it also specifies a form action. Without an action specified, the form would submit an HTTP `POST` request back to the same URL that presented the form. But here, you specify that the form should be `POST`ed to /orders (using Thymeleaf's `@{...}` operator for a context-relative path).

Therefore, you're going to need to add another method to your `OrderController` class that handles `POST` requests for /orders. You won't have a way to persist orders until the next chapter, so you'll keep it simple here—something like what you see in the next listing.

Listing 2.10 Handling a taco order submission

```
@PostMapping
public String processOrder(TacoOrder order,
      SessionStatus sessionStatus) {
 log.info("Order submitted: {}", order);
 sessionStatus.setComplete();

 return "redirect:/";
}
```

When the `processOrder()` method is called to handle a submitted order, it's given a `TacoOrder` object whose properties are bound to the submitted form fields. `TacoOrder`, much like `Taco`, is a fairly straightforward class that carries order information.

In the case of this `processOrder()` method, the `TacoOrder` object is simply logged. We'll see how to persist it to a database in the next chapter. But before `processOrder()` is done, it also calls `setComplete()` on the `SessionStatus` object passed in as a parameter. The `TacoOrder` object was initially created and placed into the session when the user created their first taco. By calling `setComplete()`, we are ensuring that the session is cleaned up and ready for a new order the next time the user creates a taco.

Now that you've developed an `OrderController` and the order form view, you're ready to try it out. Open your browser to http://localhost:8080/design, select some ingredients for your taco, and click the Submit Your Taco button. You should see a form similar to what's shown in figure 2.4.

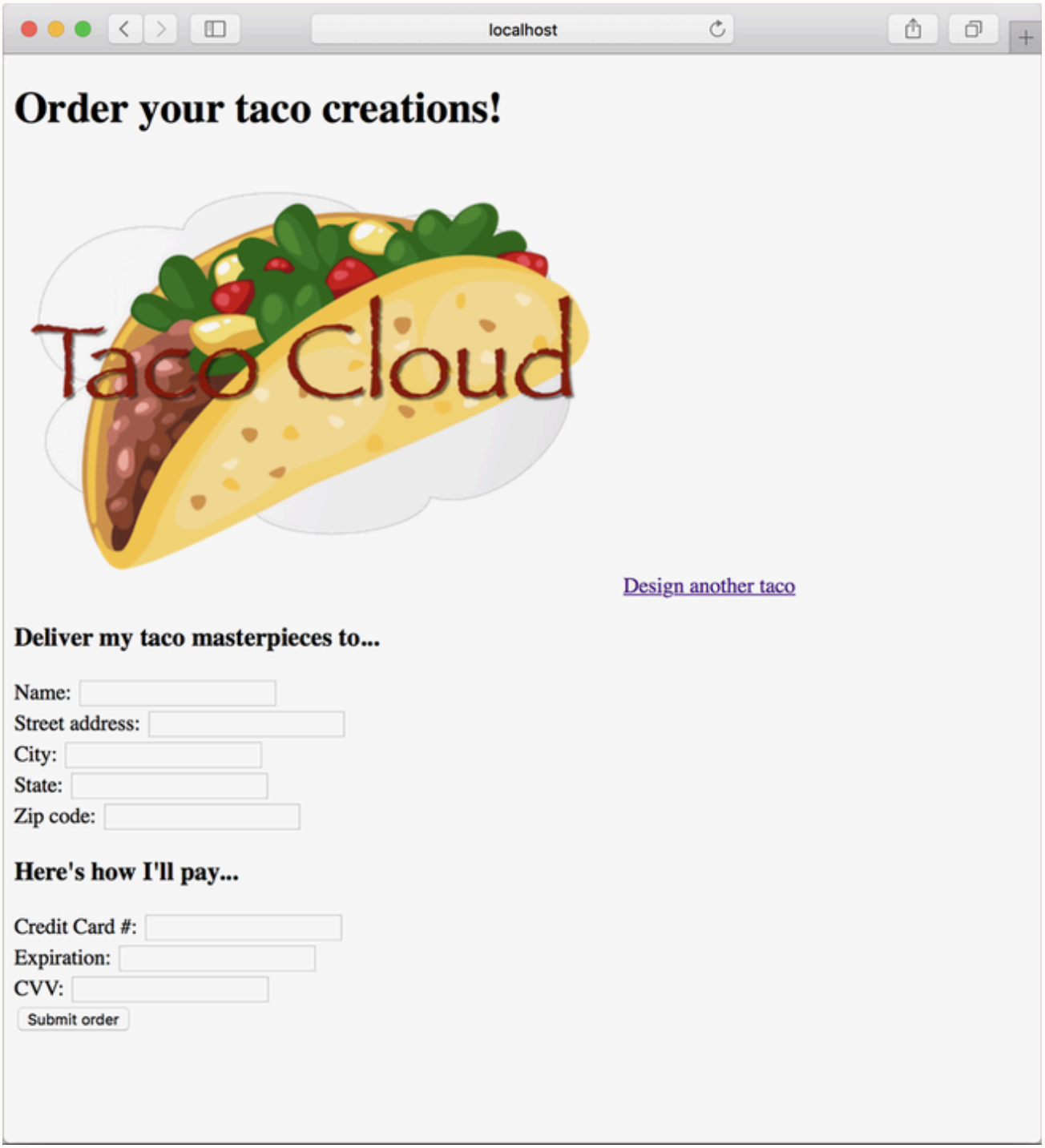


Figure 2.4 The taco order form

Fill in some fields in the form, and press the Submit Order button. As you do, keep an eye on the application logs to see your order information. When I tried it, the log entry looked something like this (reformatted to fit the width of this page):

```
Order submitted: TacoOrder(deliveryName=Craig Walls, deliveryStreet=1234 7th
Street, deliveryCity=Somewhere, deliveryState=Who knows?,
deliveryZip=zipzap, ccNumber=Who can guess?, ccExpiration=Some day,
ccCVV=See-vee-vee, tacos=[Taco(name=Awesome Sauce, ingredients=[
Ingredient(id=FLTO, name=Flour Tortilla, type=WRAP), Ingredient(id=GRBF,
name=Ground Beef, type=PROTEIN), Ingredient(id=CHED, name=Cheddar,
type=CHEESE), Ingredient(id=TMTO, name=Diced Tomatoes, type=VEGGIES),
Ingredient(id=SLSA, name=Salsa, type=SAUCE), Ingredient(id=SRCR,
```

```
name=Sour Cream, type=SAUCE)]), Taco(name=Quesoriffic, ingredients=
[Ingredient(id=FLTO, name=Flour Tortilla, type=WRAP), Ingredient(id=CHED,
name=Cheddar, type=CHEESE), Ingredient(id=JACK, name=Monterrey Jack,
type=CHEESE), Ingredient(id=TMTO, name=Diced Tomatoes, type=VEGGIES),
Ingredient(id=SRCR,name=Sour Cream, type=SAUCE)])])
```

It appears that the `processOrder()` method did its job, handling the form submission by logging details about the order. But if you look carefully at the log entry from my test order, you can see that it let a little bit of bad information get in. Most of the fields in the form contained data that couldn't possibly be correct. Let's add some validation to ensure that the data provided at least resembles the kind of information required.

2.3 Validating form input

When designing a new taco creation, what if the user selects no ingredients or fails to specify a name for their creation? When submitting the order, what if the user fails to fill in the required address fields? Or what if they enter a value into the credit card field that isn't even a valid credit card number?

As things stand now, nothing will stop the user from creating a taco without any ingredients or with an empty delivery address, or even submitting the lyrics to their favorite song as the credit card number. That's because you haven't yet specified how those fields should be validated.

One way to perform form validation is to litter the `processTaco()` and `processOrder()` methods with a bunch of `if` / `then` blocks, checking each and every field to ensure that it meets the appropriate validation rules. But that would be cumbersome and difficult to read and debug.

Fortunately, Spring supports the JavaBean Validation API (also known as JSR 303; **https://jcp.org/en/jsr/detail?id=303**). This makes it easy to declare validation rules as opposed to explicitly writing validation logic in your application code.

To apply validation in Spring MVC, you need to

- Add the Spring Validation starter to the build.
- Declare validation rules on the class that is to be validated: specifically, the `Taco` class.
- Specify that validation should be performed in the controller methods that require validation: specifically, the `DesignTacoController`'s `processTaco()` method and the `OrderController`'s `processOrder()` method.

- Modify the form views to display validation errors.

The Validation API offers several annotations that can be placed on properties of domain objects to declare validation rules. Hibernate's implementation of the Validation API adds even more validation annotations. Both can be added to a project by adding the Spring Validation starter to the build. The Validation check box under I/O in the Spring Boot Starter wizard will get the job done, but if you prefer manually editing your build, the following entry in the Maven pom.xml file will do the trick:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

Or if you're using Gradle, then this is the dependency you'll need:

```
implementation 'org.springframework.boot:spring-boot-starter-validation'
```

Is the validation starter required?

In earlier versions of Spring Boot, the Spring Validation starter was automatically included with the web starter. Starting with Spring Boot 2.3.0, you'll need to explicitly add it to your build if you intend to apply validation.

With the validation starter in place, let's see how you can apply a few annotations to validate a submitted `Taco` or `TacoOrder`.

3.1 Declaring validation rules

For the `Taco` class, you want to ensure that the `name` property isn't empty or `null` and that the list of selected ingredients has at least one item. The following listing shows an updated `Taco` class that uses `@NotNull` and `@Size` to declare those validation rules.

Listing 2.11 Adding validation to the `Taco` domain class

```
package tacos;
import java.util.List;
import javax.validation.constraints.NotNull;
import javax.validation.constraints.Size;
import lombok.Data;
```

```
@Data
public class Taco {

  @NotNull
  @Size(min=5, message="Name must be at least 5 characters long")
  private String name;
  @NotNull
  @Size(min=1, message="You must choose at least 1 ingredient")
  private List<Ingredient> ingredients;

}
```

You'll notice that in addition to requiring that the `name` property isn't `null`, you declare that it should have a value that's at least five characters in length.

When it comes to declaring validation on submitted taco orders, you must apply annotations to the `TacoOrder` class. For the address properties, you want to be sure that the user doesn't leave any of the fields blank. For that, you'll use the `@NotBlank` annotation.

Validation of the payment fields, however, is a bit more exotic. You need to ensure not only that the `ccNumber` property isn't empty but also that it contains a value that could be a valid credit card number. The `ccExpiration` property must conform to a format of MM/YY (two-digit month and year), and the `ccCVV` property needs to be a three-digit number. To achieve this kind of validation, you need to use a few other JavaBean Validation API annotations and borrow a validation annotation from the Hibernate Validator collection of annotations. The following listing shows the changes needed to validate the `TacoOrder` class.

Listing 2.12 Validating order fields

```
package tacos;
import javax.validation.constraints.Digits;
import javax.validation.constraints.NotBlank;
import javax.validation.constraints.Pattern;
import org.hibernate.validator.constraints.CreditCardNumber;
import java.util.List;
import java.util.ArrayList;
import lombok.Data;

@Data
```

```
public class TacoOrder {

  @NotBlank(message="Delivery name is required")
  private String deliveryName;

  @NotBlank(message="Street is required")
  private String deliveryStreet;

  @NotBlank(message="City is required")
  private String deliveryCity;

  @NotBlank(message="State is required")
  private String deliveryState;

  @NotBlank(message="Zip code is required")
  private String deliveryZip;

  @CreditCardNumber(message="Not a valid credit card number")
  private String ccNumber;

  @Pattern(regexp="^(0[1-9]|1[0-2])([\\/])([2-9][0-9])$",
         message="Must be formatted MM/YY")
  private String ccExpiration;

  @Digits(integer=3, fraction=0, message="Invalid CVV")
  private String ccCVV;

  private List<Taco> tacos = new ArrayList<>();

  public void addTaco(Taco taco) {
    tacos.add(taco);
  }
}
```

As you can see, the `ccNumber` property is annotated with `@CreditCardNumber`. This annotation declares that the property's value must be a valid credit card number that passes the Luhn algorithm check (**https://creditcardvalidator.org/articles/luhn-algorithm**). This prevents user mistakes and deliberately bad data but doesn't guarantee that the credit card number is actually assigned to an account or that the account can be used for charging.

Unfortunately, there's no ready-made annotation for validating the MM/YY format of the `ccExpiration` property. I've applied the `@Pattern` annotation, providing it with a regular expression that ensures that the property value adheres to the desired format. If

you're wondering how to decipher the regular expression, I encourage you to check out the many online regular expression guides, including **http://www.regular-expressions.info/**. Regular expression syntax is a dark art and certainly outside the scope of this book. Finally, we annotate the `ccCVV` property with `@Digits` to ensure that the value contains exactly three numeric digits.

All of the validation annotations include a `message` attribute that defines the message you'll display to the user if the information they enter doesn't meet the requirements of the declared validation rules.

3.2 Performing validation at form binding

Now that you've declared how a `Taco` and `TacoOrder` should be validated, we need to revisit each of the controllers, specifying that validation should be performed when the forms are `POST`ed to their respective handler methods.

To validate a submitted `Taco`, you need to add the JavaBean Validation API's `@Valid` annotation to the `Taco` argument of the `DesignTacoController`'s `processTaco()` method, as shown next.

Listing 2.13 Validating a `POST`ed `Taco`

```
import javax.validation.Valid;
import org.springframework.validation.Errors;

...

 @PostMapping
 public String processTaco(
       @Valid Taco taco, Errors errors,
       @ModelAttribute TacoOrder tacoOrder) {

  if (errors.hasErrors()) {
   return "design";
  }

  tacoOrder.addTaco(taco);
  log.info("Processing taco: {}", taco);

  return "redirect:/orders/current";
 }
```

The `@Valid` annotation tells Spring MVC to perform validation on the submitted `Taco` object after it's bound to the submitted form data and before the `processTaco()` method is called. If there are any validation errors, the details of those errors will be captured in an `Errors` object that's passed into `processTaco()`. The first few lines of `processTaco()` consult the `Errors` object, asking its `hasErrors()` method if there are any validation errors. If there are, the method concludes without processing the `Taco` and returns the `"design"` view name so that the form is redisplayed.

To perform validation on submitted `TacoOrder` objects, similar changes are also required in the `processOrder()` method of `OrderController`, as shown in the next code listing.

Listing 2.14 Validating a `POST`ed `TacoOrder`

```
@PostMapping
public String processOrder(@Valid TacoOrder order, Errors errors,
      SessionStatus sessionStatus) {
  if (errors.hasErrors()) {
    return "orderForm";
  }

  log.info("Order submitted: {}", order);
  sessionStatus.setComplete();

  return "redirect:/";
}
```

In both cases, the method will be allowed to process the submitted data if there are no validation errors. If there are validation errors, the request will be forwarded to the form view to give the user a chance to correct their mistakes.

But how will the user know what mistakes require correction? Unless you call out the errors on the form, the user will be left guessing about how to successfully submit the form.

2.3.3 Displaying validation errors

Thymeleaf offers convenient access to the `Errors` object via the `fields` property and with its `th:errors` attribute. For example, to display validation errors on the credit card number field, you can add a `<span>` element that uses these error references to the order form template, as follows.

Listing 2.15 Displaying validation errors

```
<label for="ccNumber">Credit Card #: </label>
    <input type="text" th:field="*{ccNumber}"/>
    <span class="validationError"
        th:if="${#fields.hasErrors('ccNumber')}"
        th:errors="*{ccNumber}">CC Num Error</span>
```

Aside from a `class` attribute that can be used to style the error so that it catches the user's attention, the `<span>` element uses a `th:if` attribute to decide whether to display the `<span>`. The `fields` property's `hasErrors()` method checks whether there are any errors in the `ccNumber` field. If so, the `<span>` will be rendered.

The `th:errors` attribute references the `ccNumber` field and, assuming errors exist for that field, it will replace the placeholder content of the `<span>` element with the validation message.

If you were to sprinkle similar `<span>` tags around the order form for the other fields, you might see a form that looks like figure 2.5 when you submit invalid information. The errors indicate that the name, city, and ZIP code fields have been left blank and that all of the payment fields fail to meet the validation criteria.

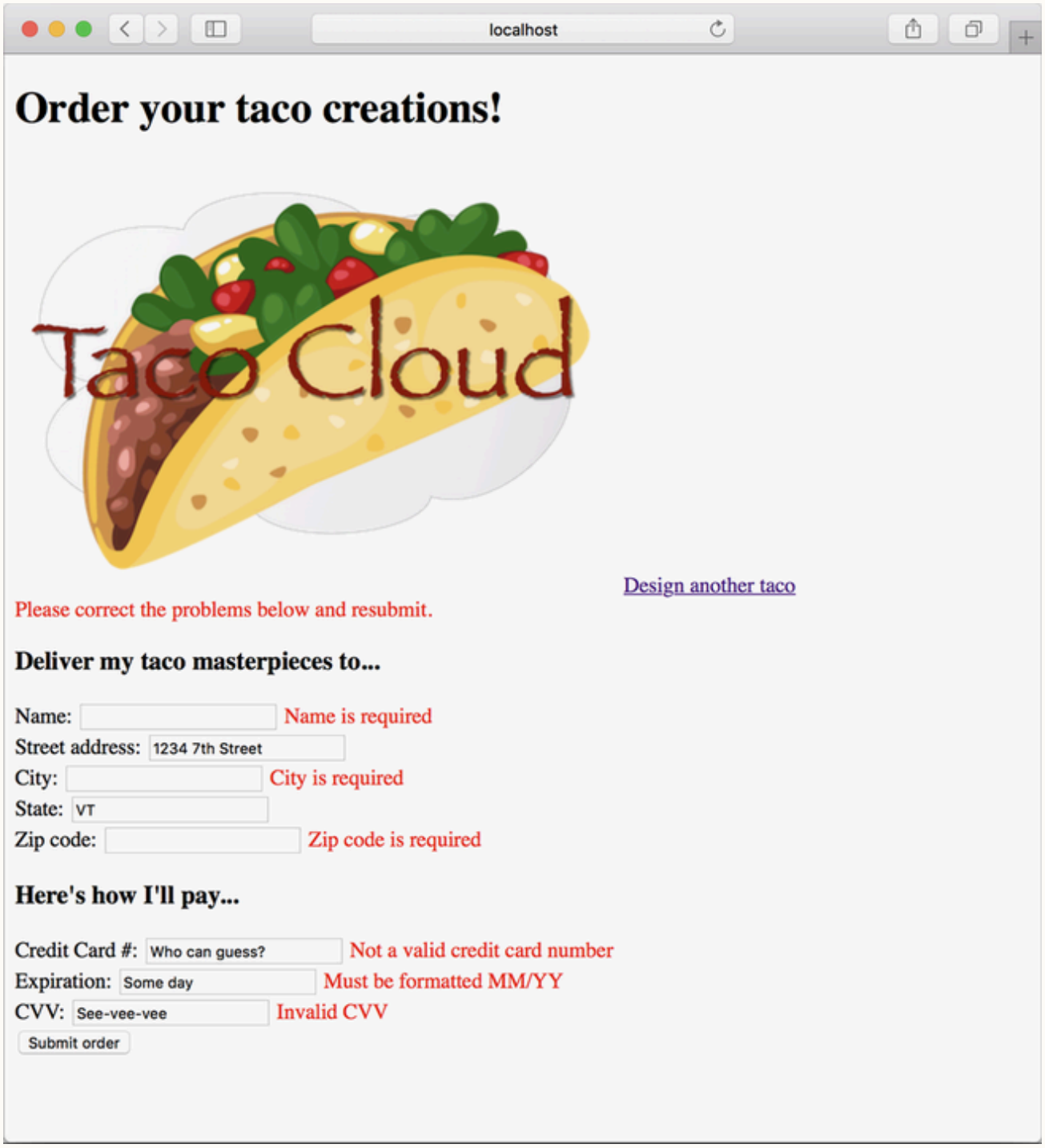


Figure 2.5 Validation errors displayed on the order form

Now your Taco Cloud controllers not only display and capture input, but they also validate that the information meets some basic validation rules. Let's step back and reconsider the `HomeController` from chapter 1, looking at an alternative implementation.

2.4 Working with view controllers

Thus far, you've written three controllers for the Taco Cloud application. Although each controller serves a distinct purpose in the functionality of the application, they all pretty much adhere to the following programming model:

- They're all annotated with `@Controller` to indicate that they're controller classes that should be automatically discovered by Spring component scanning and instantiated as beans in the Spring application context.
- All but `HomeController` are annotated with `@RequestMapping` at the class level to define a baseline request pattern that the controller will handle.
- They all have one or more methods that are annotated with `@GetMapping` or `@PostMapping` to provide specifics on which methods should handle which kinds of requests.

Most of the controllers you'll write will follow that pattern. But when a controller is simple enough that it doesn't populate a model or process input—as is the case with your `HomeController`—there's another way that you can define the controller. Have a look at the next listing to see how you can declare a view controller—a controller that does nothing but forward the request to a view.

Listing 2.16 Declaring a view controller

```
package tacos.web;

import org.springframework.context.annotation.Configuration;
import org.springframework.web.servlet.config.annotation.ViewControllerRegistry;
import org.springframework.web.servlet.config.annotation.WebMvcConfigurer;

@Configuration
public class WebConfig implements WebMvcConfigurer {

  @Override
  public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("home");
  }

}
```

The most significant thing to notice about `WebConfig` is that it implements the `WebMvcConfigurer` interface. `WebMvcConfigurer` defines several methods for configuring Spring MVC. Even though it's an interface, it provides default implementations of all the methods, so you need to override only the methods you need. In this case, you override `addViewControllers()`.

The `addViewControllers()` method is given a `ViewControllerRegistry` that you can use to register one or more view controllers. Here, you call `addViewController()` on the reg-

istry, passing in "/", which is the path for which your view controller will handle `GET` requests. That method returns a `ViewControllerRegistration` object, on which you immediately call `setViewName()` to specify `home` as the view that a request for "/" should be forwarded to.

And just like that, you've been able to replace `HomeController` with a few lines in a configuration class. You can now delete `HomeController`, and the application should still behave as it did before. The only other change required is to revisit `HomeControllerTest` from chapter 1, removing the reference to `HomeController` from the `@WebMvcTest` annotation, so that the test class will compile without errors.

Here, you've created a new `WebConfig` configuration class to house the view controller declaration. But any configuration class can implement `WebMvcConfigurer` and override the `addViewController` method. For instance, you could have added the same view controller declaration to the bootstrap `TacoCloudApplication` class like this:

```
@SpringBootApplication
public class TacoCloudApplication implements WebMvcConfigurer {

  public static void main(String[] args) {
    SpringApplication.run(TacoCloudApplication.class, args);
  }
  @Override
  public void addViewControllers(ViewControllerRegistry registry) {
    registry.addViewController("/").setViewName("home");
  }

}
```

By extending an existing configuration class, you can avoid creating a new configuration class, keeping your project artifact count down. But I prefer creating a new configuration class for each kind of configuration (web, data, security, and so on), keeping the application bootstrap configuration clean and simple.

Speaking of view controllers—and more generically, the views that controllers forward requests to—so far you've been using Thymeleaf for all of your views. I like Thymeleaf a lot, but maybe you prefer a different template model for your application views. Let's have a look at Spring's many supported view options.

2.5 Choosing a view template library

For the most part, your choice of a view template library is a matter of personal taste. Spring is flexible and supports many common templating options. With only a few small exceptions, the template library you choose will itself have no idea that it's even working with Spring.[3]

Table 2.2 catalogs the template options supported by Spring Boot autoconfiguration.

Table 2.2 Supported template options

Template	Spring Boot starter dependency
FreeMarker	spring-boot-starter-freemarker
Groovy templates	spring-boot-starter-groovy-templates
JavaServer Pages (JSP)	None (provided by Tomcat or Jetty)
Mustache	spring-boot-starter-mustache
Thymeleaf	spring-boot-starter-thymeleaf

Generally speaking, you select the view template library you want, add it as a dependency in your build, and start writing templates in the /templates directory (under the src/main/resources directory in a Maven or Gradle project). Spring Boot detects your chosen template library and automatically configures the components required for it to serve views for your Spring MVC controllers.

You've already done this with Thymeleaf for the Taco Cloud application. In chapter 1, you selected the Thymeleaf check box when initializing the project. This resulted in Spring Boot's Thymeleaf starter being included in the pom.xml file. When the application starts up, Spring Boot autoconfiguration detects the presence of Thymeleaf and automatically configures the Thymeleaf beans for you. All you had to do was start writing templates in /templates.

If you'd rather use a different template library, you simply select it at project initialization or edit your existing project build to include the newly chosen template library.

For example, let's say you wanted to use Mustache instead of Thymeleaf. No problem. Just visit the project pom.xml file and replace this

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-thymeleaf</artifactId>
</dependency>
```

with this:

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-mustache</artifactId>
</dependency>
```

Of course, you'd need to make sure that you write all the templates with Mustache syntax instead of Thymeleaf tags. The specifics of working with Mustache (or any of the template language choices) is well outside of the scope of this book, but to give you an idea of what to expect, here's a snippet from a Mustache template that will render one of the ingredient groups in the taco design form:

```
<h3>Designate your wrap:</h3>
{{#wrap}}
<div>
  <input name="ingredients" type="checkbox" value="{{id}}" />
  <span>{{name}}</span><br/>
</div>
{{/wrap}}
```

This is the Mustache equivalent of the Thymeleaf snippet in section 2.1.3. The `{{#wrap}}` block (which concludes with `{{/wrap}}`) iterates through a collection in the request attribute whose key is `wrap` and renders the embedded HTML for each item. The `{{id}}` and `{{name}}` tags reference the `id` and `name` properties of the item (which should be an `Ingredient`).

You'll notice in table 2.2 that JSP doesn't require any special dependency in the build. That's because the servlet container itself (Tomcat by default) implements the JSP specification, thus requiring no further dependencies.

But there's a gotcha if you choose to use JSP. As it turns out, Java servlet containers—including embedded Tomcat and Jetty containers—usually look for JSPs somewhere under /WEB-INF. But if you're building your application as an executable JAR file, there's no way to satisfy that requirement. Therefore, JSP is an option only if you're building your application as a WAR file and deploying it in a traditional servlet container. If you're building an executable JAR file, you must choose Thymeleaf, FreeMarker, or one of the other options in table 2.2.

2.5.1 Caching templates

By default, templates are parsed only once—when they're first used—and the results of that parse are cached for subsequent use. This is a great feature for production, because it prevents redundant template parsing on each request and thus improves performance.

That feature is not so awesome at development time, however. Let's say you fire up your application, hit the taco design page, and decide to make a few changes to it. When you refresh your web browser, you'll still be shown the original version. The only way you can see your changes is to restart the application, which is quite inconvenient.

Fortunately, we have a way to disable caching. All we need to do is set a template-appropriate caching property to `false`. Table 2.3 lists the caching properties for each of the supported template libraries.

Table 2.3 Properties to enable/disable template caching

Template	Cache-enable property
FreeMarker	spring.freemarker.cache
Groovy templates	spring.groovy.template.cache
Mustache	spring.mustache.cache
Thymeleaf	spring.thymeleaf.cache

By default, all of these properties are set to `true` to enable caching. You can disable caching for your chosen template engine by setting its cache property to `false`. For example, to disable Thymeleaf caching, add the following line in application.properties:

```
spring.thymeleaf.cache=false
```

The only catch is that you'll want to be sure to remove this line (or set it to `true`) before you deploy your application to production. One option is to set the property in a profile. (We'll talk about profiles in chapter 6.)

A much simpler option is to use Spring Boot's DevTools, as we opted to do in chapter 1. Among the many helpful bits of development-time help offered by DevTools, it will disable caching for all template libraries but will disable itself (and thus reenable template caching) when your application is deployed.

Summary

- Spring offers a powerful web framework called Spring MVC that can be used to develop the web frontend for a Spring application.
- Spring MVC is annotation-based, enabling the declaration of request-handling methods with annotations such as `@RequestMapping`, `@GetMapping`, and `@PostMapping`.
- Most request-handling methods conclude by returning the logical name of a view, such as a Thymeleaf template, to which the request (along with any model data) is forwarded.
- Spring MVC supports validation through the JavaBean Validation API and implementations of the Validation API such as Hibernate Validator.
- View controllers can be registered with `addViewController` in a `WebMvcConfigurer` class to handle HTTP `GET` requests for which no model data or processing is required.
- In addition to Thymeleaf, Spring supports a variety of view options, including FreeMarker, Groovy templates, and Mustache.

---

[1] For a much more in-depth discussion of application domains, I suggest Eric Evans's *Domain-Driven Design* (Addison-Wesley Professional, 2003).

[2] The contents of the stylesheet aren't relevant to our discussion; it contains only styling to present the ingredients in two columns instead of one long list of ingredients.

[3] One such exception is Thymeleaf's Spring Security dialect, which we'll talk about in chapter 5.

‹ 1 Getting started with Spring
3 Working with data ›